

OriPics GPSA v2.2

OriPics — Generator Product Security Architecture Document

Applicant: SantaHades Co., Ltd. **Record ID:** 019e4988-9d7c-72f8-8675-22eacf3e1904 **Version:** 2.2 (2026-05-29)
Supersedes: v2.1 (2026-05-28), v2.0 (2026-05-24), v1.0 (2026-05-22) **Target Assurance Level:** Level 1

Summary of Changes in v2.2

Item	v2.1	v2.2
c2pa.actions.v2 assertion placement	gathered_assertions (via addAssertion()) — nonconformant	created_assertions (via builder.addAction() per action) — conformant
Web client in TOE boundary	Included as Edge subsystem	Explicitly excluded from TOE — Web is content-submission portal only; Req 6.2.1.5/6 satisfied by mobile clients (App Attest/Play Integrity)
Action for Standard tier (web/mobile file pick)	c2pa.created + digitalCapture — incorrect (capture not verifiable)	c2pa.created (no digitalSourceType); only Verified tier (mobile camera, App Attest/Play Integrity) uses c2pa.created + digitalCapture
Signing key storage language	“SSL.com eSigner Cloud HSM (production)” — tentative/unimplemented	Clearly stated: current production = Vercel encrypted env vars (AES-256); HSM is planned Phase 2 post-conformance
TOE for Req 6.2.1.5/6	Web + Mobile	Mobile only (iOS App Attest + Android Play Integrity); Web explicitly excluded

Summary of Changes in v2.1

Item	v2.0	v2.1
TLS for Backend→Supabase	TLS 1.2+	TLS 1.3 enforced via instrumentation.ts tls.DEFAULT_MIN_VERSION
Pre-existing manifest handling (§C.2.7)	“Pending guidance” (not implemented)	Approach A implemented: c2pa.opened + parentOf ingredient
Action for ingested file	c2pa.created (incorrect)	c2pa.opened when prior manifest detected
Assertion classification note	“All in created_assertions” (incorrect)	Corrected: c2pa-rs Claim v2 places hash in created, data in gathered
Attachments	Standard sample only	+ingredient sample (Pixel Camera JPG from Interoperability Library)

Summary of Changes in v2.0

Item	v1.0	v2.0
Implementation Class	Backend	Distributed
TOE Architecture	Backend-only diagram	Distributed diagram (3 Edge clients + Backend)
Edge-to-Backend mutual auth	Not documented	Documented (§C.2.2 §4)
Digital Source Type	algorithmicMedia (incorrect)	digitalCapture
Capture-only / Ingredient policy	Not documented	New §C.2.7

C.1. Generator Product Information

C.1.1. Applicant Organization Details

- **Legal Name:** SantaHades Co., Ltd. (주식회사 산타하데스)
- **Address:** #B01-H306, Terrace Garden, 150-29 Gongse-ro, Giheung-gu, Yongin-si, Gyeonggi-do 17084, Republic of Korea
- **Contact Person:** Yongseog Son, Representative Director and CEO
- **Email:** hi@ori.pics
- **Phone:** +82-10-3717-9717
- **Security Contact:** security@ori.pics (RFC 9116 security.txt published at <https://www.ori.pics/.well-known/security.txt>)

C.1.2. Distinguished Name

Field	Value
Common Name (CN)	OriPics
Organization (O)	SantaHades Co., Ltd.
Organizational Unit (OU)	<i>(not applicable — single-team organization)</i>
Country (C)	KR

C.1.3. Generator Product Description

OriPics is a photo provenance and authenticity platform for the Korean consumer and SMB market, with planned expansion to global users. It allows users to prove that an image is original (untampered) and to certify when and where it was captured.

Platform coverage and TOE classification:

Platform	TOE membership	Role
OriPics iOS	Within TOE	Native camera capture (Verified tier, App Attest) + file pick (Standard tier). Device integrity: Apple App Attest.
OriPics Android	Within TOE	Native camera capture (Verified tier, Play Integrity) + file pick (Standard tier). Device integrity: Google Play Integrity.
OriPics Web	Outside TOE	Browser-based content submission portal only. Browser JS cannot provide cryptographic subsystem authentication. Web uploads receive c2pa.published action (no digitalCapture assertion). Not a conformant Edge subsystem.

TOE boundary justification: The C2PA Generator Product TOE requires that Edge subsystems be mutually authenticated with the Backend (Req 6.2.1.5/6). The mobile clients satisfy this requirement via Apple App Attest (iOS) and Google Play Integrity (Android). The Web client runs in a browser JavaScript environment that provides no secure storage or execution boundary; it cannot be cryptographically authenticated as a subsystem instance. Accordingly, the Web client is **explicitly excluded from the TOE** for C2PA manifest generation purposes. Web uploads are accepted as user-submitted content; the Backend issues `c2pa.published` claims (not `c2pa.created`) for these assets.

The mobile and backend clients share a common backend signing service hosted on Vercel Functions. The backend holds the C2PA signing certificate; no signing key material is present on any edge device.

Note on separate submissions: Per C2PA Conformance Program requirements, OriPics iOS and OriPics Android will be submitted as separate generator products with separate Intake Forms. This document covers the full Distributed architecture shared by all platforms. The backend signing service and certificate are common to all platforms; platform-specific details are called out where relevant.

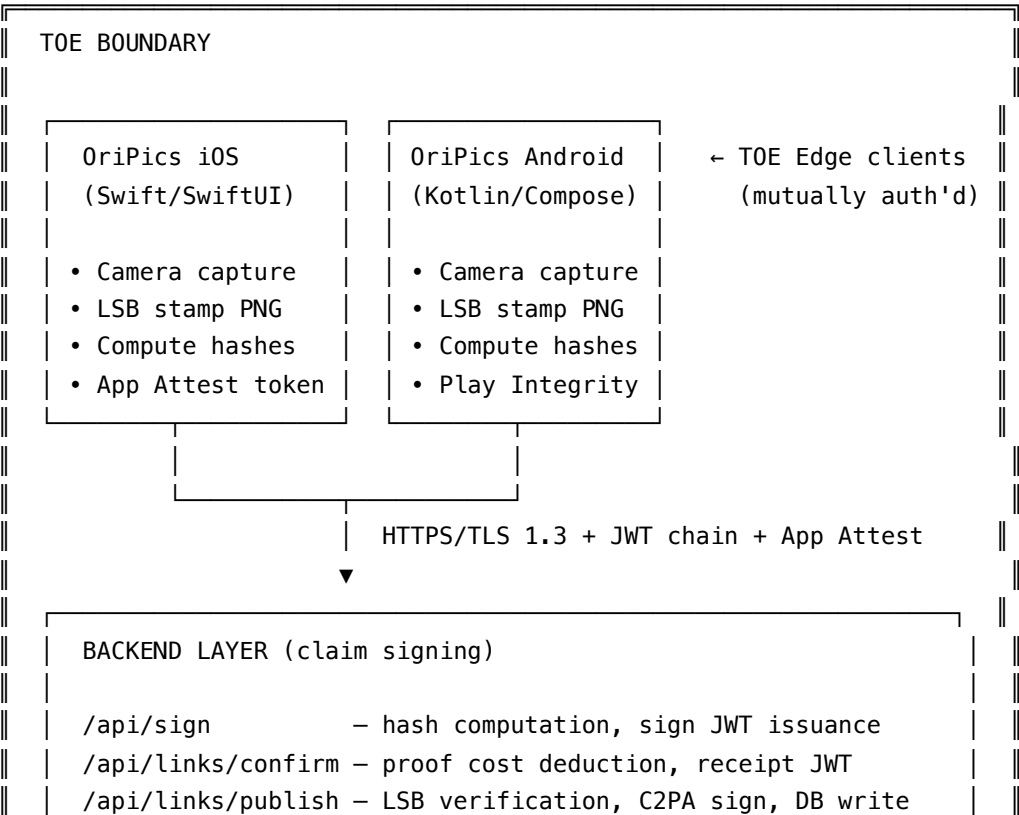
Intended use cases: - Traffic accident scene documentation for insurance and legal submissions - Real estate move-in/move-out condition evidence - Creator portfolio originality proof - Media and journalism photo provenance - Dating and social media identity verification

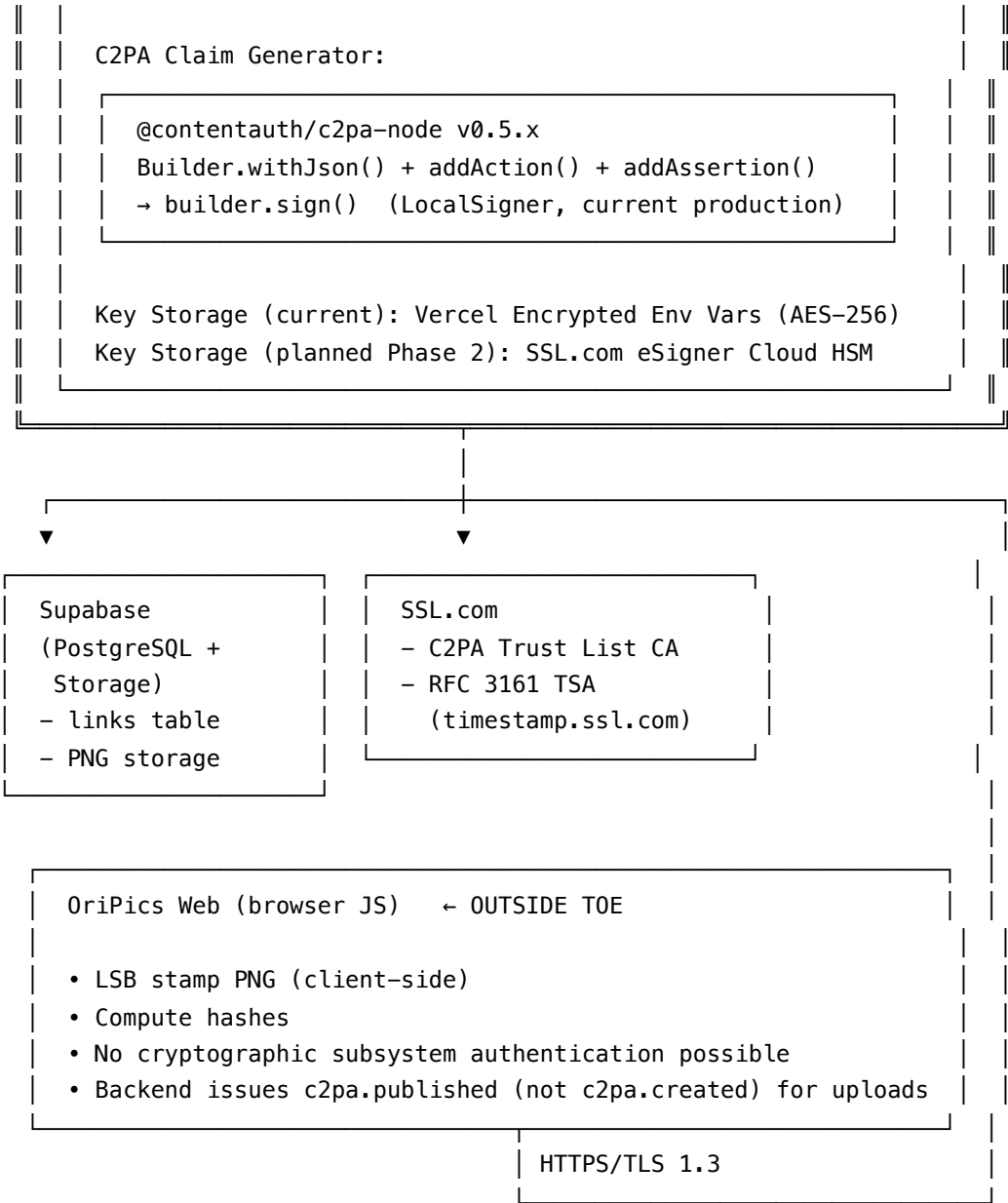
Key features: - Steganographic pixel-integrity stamping (proprietary LSB embedding + HMAC verification) - C2PA manifest generation and attachment to output PNG images - Two trust tiers: Standard (web upload or mobile file pick) and Verified (mobile native camera capture with device attestation) - Preservation of provenance chain: pre-existing C2PA manifests are included as `c2pa.ingredient` assertions - Public shareable links with 7-day (Free) or permanent (Pro/Business) retention

Target audience: Individual consumers (B2C, traffic accidents, personal proof) and small businesses (B2B, real estate agencies, media organizations).

C.1.4. Generator Product Target of Evaluation (TOE) Description

The OriPics Generator Product TOE uses a **Distributed Implementation Class**. Claim preparation happens on Edge devices; claim signing happens on the Backend service.





Platform runtime: - Backend: Vercel Serverless Functions (AWS-backed), Node.js 18+ runtime - Edge (Web): Browser JavaScript (Chrome, Safari, Firefox — all modern browsers) - Edge (iOS): Swift/SwiftUI, iOS 16+, ARM64 - Edge (Android): Kotlin/Jetpack Compose, Android 11+, ARM64

Third-party services in TOE scope:

Service	Role	Security Properties
Vercel	Hosting, serverless compute, CDN, TLS termination	SOC 2 Type II, encrypted env vars, immutable deployments
Supabase	PostgreSQL database, object storage (S3-compatible)	SOC 2 Type II, RLS policies, AES-256 encryption at rest
SSL.com	C2PA Trust List CA, eSigner Cloud HSM, TSA	C2PA Trust List registered CA (SSL.com C2PA ECC/RSA Root CA 2025), WebTrust audited
GitHub	Source code repository, CI/CD	Branch protection, Dependabot, CodeQL, secret scanning

Service	Role	Security Properties
Apple App Attest	iOS device integrity attestation (Verified tier)	Apple-managed PKI, attestation key bound to device Secure Enclave
Google Play Integrity	Android device integrity attestation (Verified tier)	Google-managed, Play Integrity API

C.1.5. Implementation Class

Distributed

The OriPics Generator Product is classified as **Distributed** because claim generation spans both Edge and Backend subsystems:

Subsystem	TOE	Role in Claim Generation
Edge — OriPics iOS	In TOE	Native camera capture; steganographic LSB stamping; HMAC hash computation; App Attest token for Verified tier; PNG upload via signed URL
Edge — OriPics Android	In TOE	Same as iOS; Google Play Integrity for Verified tier
Edge — OriPics Web	Outside TOE	Content submission portal only. LSB stamping and hash computation performed client-side, but no cryptographic subsystem authentication possible. Backend issues <code>c2pa.published</code> for all Web uploads. Receives hashes from Edge via HTTPS; verifies mobile Edge identity via JWT chain + LSB hash extraction + App Attest/Play Integrity; constructs C2PA manifest; signs manifest using <code>@contentauth/c2pa-node</code> ; writes to Supabase
Backend — Vercel Functions	In TOE	

Signing key location: The C2PA signing private key is held exclusively by the Backend. **Current production:** Vercel Encrypted Environment Variables (AES-256). **Planned Phase 2:** SSL.com eSigner Cloud HSM via CSC API (not yet implemented as of v2.2). No signing key material is present on any Edge device.

Claim generator values: - Standard tier (all platforms): `oripics/0.1.0` - Verified tier (mobile native capture): `oripics/0.1.0 (mobile-native)`

C.1.6. Target Max Assurance Level

Level 1

C.1.7. Target Generator Product Capabilities

Claim generation: - Still image media types: - `image/png`

Claim validation: None (Generator Product only; validation is delegated to external C2PA-compatible tools such as `contentcredentials.org/verify`).

C.2. Security Architecture Details

C.2.1. Authentication for Certificate Enrollment

Certificate Enrollment Process

OriPics uses SSL.com as its Certification Authority (CA), which is on the C2PA Trust List.

Current (pre-production / sandbox): 1. CSR generated with OpenSSL (ECC P-256) 2. CSR submitted to SSL.com c2pasign.com sandbox for test cert issuance 3. Test cert + private key stored as Vercel encrypted environment variables 4. LocalSigner in @contentauth/c2pa-node uses the cert+key to sign manifests

Production (post-Conformance Letter): 1. SSL.com issues a C2PA Claim Signing Certificate to OriPics after identity verification (OV-level KYC) 2. Private key is generated and stored within SSL.com eSigner Cloud HSM — the private key never leaves the HSM 3. OriPics backend authenticates to SSL.com eSigner CSC API using OAuth 2.0 client credentials (client_id + client_secret) 4. Each signing operation is a remote API call: the backend sends the hash to be signed, SSL.com HSM signs it, and returns the signature 5. Certificate rotation: SSL.com manages certificate lifecycle; OriPics updates Vercel environment variables when a new cert is issued

Management of Certificate Enrollment Authentication Secrets

(Required for Assurance Level 1)

Authentication method: OAuth 2.0 client credentials (client_id + client_secret) for SSL.com eSigner CSC API.

Secret management: - ORIPICS_ESIGNER_CLIENT_ID and ORIPICS_ESIGNER_CLIENT_SECRET are stored in Vercel encrypted environment variables - Vercel encrypts environment variables at rest using AES-256 and restricts access via team RBAC - Environment variables are scoped to Production environment only (Preview/Development use separate sandbox credentials) - Secrets are never committed to source code; .env files are gitignored - GitHub secret scanning is enabled on the repository to detect accidental credential exposure - Access to Vercel project settings (where env vars are managed) requires team owner authentication with MFA enabled

C.2.2. Key Generation, Storage, and Usage

Key Generation and Storage Method

(Required for Assurance Level 1)

1. Key generation and storage:

- **Algorithm:** ECDSA with P-256 (ES256), as required by C2PA specification
- **Current production:** Key generated locally via OpenSSL, stored as PEM in Vercel encrypted environment variable (ORIPICS_C2PA_KEY_PEM). Vercel encrypts environment variables at rest with AES-256. The key is decrypted at function cold start, held in Lambda memory only for the duration of the signing operation, and not persisted to disk. This satisfies Level 1 Req 6.2.1.1 (encrypted storage).
- **Planned Phase 2 (not yet implemented):** Migrate to SSL.com eSigner Cloud HSM (FIPS 140-2 Level 3). Private key would be generated within the HSM and never leave it. OriPics code would call the CSC API for signing operations. This enhancement is planned post-conformance and is not evaluated in this submission.

2. Key access controls:

- Only the Vercel Functions runtime in the Production environment can access the signing credentials
- Vercel environment variables are encrypted at rest (AES-256) and in transit (TLS 1.3)
- Access to the key PEM in Vercel is restricted to the single Vercel team owner (MFA-protected); no other team members exist
- Application code accesses the key only within the /api/links/publish route handler; no other code path reads the signing key
- No signing key material is present on any Edge device (Web browser, iOS, Android)

3. Key rotation process:

- **Sandbox:** Manual rotation by generating a new key pair and updating Vercel env vars

- **Production (SSL.com HSM):** Certificate lifecycle managed by SSL.com. Upon certificate expiration or revocation, OriPics updates the credential ID in Vercel env vars. The old key is destroyed within the HSM by SSL.com per their key management policy
- Key rotation can be performed with zero downtime by deploying updated env vars to Vercel (atomic deployment model)

4. Edge-to-Backend mutual authentication (Distributed Implementation Class):

TOE scope: Req 6.2.1.5/6 (mutual authentication of Edge and Backend subsystems) applies **only to the mobile Edge clients (iOS, Android)** which are within the TOE. The Web client is explicitly outside the TOE and is not subject to subsystem mutual authentication requirements. The Backend accepts Web uploads but issues `c2pa.published` claims (not `c2pa.created + digitalCapture`) for those assets, consistent with its content-submission-only role.

The OriPics Distributed architecture uses a three-stage JWT chain to authenticate the **mobile** Edge client to the Backend signing service. This mechanism verifies that: (a) the user is authenticated (NextAuth session); and (b) the PNG uploaded to Storage was produced by a legitimate OriPics Edge client (LSB hash verification).

Stage 1 — /api/sign (Backend): 1. Validates NextAuth session; checks credit balance 2. Computes `final_hash = HMAC-SHA256(salt, meta_bytes || inner_hash || border_hash)` where `inner_hash` and `border_hash` are computed by the Edge from the original image pixels 3. Issues a sign JWT (`aud: links/confirm`, TTL 5 minutes) containing `{ user_id, tier, link_id, storage_path, timestamp, width, height, final_hash_hex }` 4. Returns `final_hash` (hex) and a Supabase signed upload URL to the Edge

Stage 2 — Edge (all clients): 1. Embeds `meta_bytes || final_hash` into the PNG border pixels via LSB steganography 2. Uploads the stamped PNG to Supabase Storage using the signed upload URL 3. Calls `/api/links/confirm` with the sign JWT

Stage 3 — /api/links/confirm (Backend): 1. Verifies sign JWT signature, expiry, and audience 2. Deducts proof credits 3. Issues a receipt JWT (`aud: links/publish`, TTL 30 days) containing all sign JWT claims including `final_hash_hex`

Stage 4 — /api/links/publish (Backend): 1. Verifies receipt JWT signature, expiry, and audience 2. Verifies `claims.user_id === sessionUserId` (prevents cross-user replay) 3. Downloads the stamped PNG from Supabase Storage 4. **LSB hash extraction and verification:** Decodes PNG pixels using `pngjs`, reads border-pixel LSBs using `extractPayloadV4()`, extracts embedded `final_hash` from the payload. Compares against `final_hash_hex` from the receipt JWT using `crypto.timingSafeEqual()`. Mismatch → 422 response + credit refund 5. Only if hash matches: constructs and signs the C2PA manifest

This mechanism ensures that a tampered PNG (one where pixel content was swapped after the sign JWT was issued but before publish) will fail LSB verification and be rejected before C2PA signing.

Backend-to-SSL.com authentication: - OriPics Backend authenticates to SSL.com eSigner CSC API using OAuth 2.0 client credentials - SSL.com authenticates the backend by validating the client credentials against the registered OriPics account - All communication between the backend and SSL.com is over HTTPS/TLS 1.3

Mobile Edge device attestation (Verified tier): - iOS: Edge client obtains an Apple App Attest attestation token (`DCAppAttestService`), bound to the device Secure Enclave key and a server-issued nonce. Backend verifies the attestation chain against Apple's root CA - Android: Edge client obtains a Google Play Integrity verdict token, bound to the app's Play Store identity and a server-issued nonce. Backend verifies the token against Google's Play Integrity API - Attestation token hash is stored in the `com.oripics.verified` C2PA assertion for audit

C.2.3. Protections Against Claim Generator Misconfiguration and Abuse

Processes for Detecting Vulnerabilities in Dependencies (SCA/SBOM)

(Required for Assurance Level 1)

1. SCA/SBOM dependency vulnerability scanning tools:

- **GitHub Dependabot:** Enabled on the repository (SantaHades/OriPics-MVP-Front). Monitors all npm dependencies for known CVEs from the GitHub Advisory Database (NIST NVD data). Configured via `.github/dependabot.yml`:
 - Major version updates: blocked (manual review required)
 - Minor/patch updates: grouped weekly PRs
 - Security updates: automatic PRs for CRITICAL and HIGH severity
- **GitHub CodeQL:** Enabled for JavaScript/TypeScript static analysis. Runs on every PR and weekly scheduled scans. Detects OWASP Top 10 vulnerabilities (SQL injection, XSS, command injection, path traversal, etc.)
- **GitHub Secret Scanning:** Enabled to detect accidental credential commits (API keys, certificates, tokens)
- **npm audit:** Run as part of the development workflow; `pnpm audit` checks against the npm advisory database

2. Build and deployment pipeline vulnerability prevention:

- Dependabot automatically creates PRs for CRITICAL and HIGH severity vulnerabilities
- The development team triages vulnerability reports within 7 days of detection
- CRITICAL and HIGH vulnerabilities are fixed or mitigated within 90 days (C2PA Security Requirements §6.3.1)
- Vercel deployments are immutable: each deployment is a frozen snapshot of the code at a specific Git commit. Once deployed, the running code cannot be modified
- GitHub branch protection on `main` requires PR review and CI passage before merge
- Historical evidence: 21 Dependabot advisories triaged on 2026-05-09; Next.js 14.1.0 → 14.2.35 security patch applied (16 advisories resolved); `next-intl` prototype pollution fix applied 2026-05-10

C.2.4. Protections Against Misconfiguration and Abuse of Software That Processes or Modifies Digital Content or Assertions

(Required for Assurance Level 1)

The software that processes and/or modifies Digital Content in the OriPics TOE includes:

1. **Client-side steganographic stamping** (`oripics-stamp` library, runs on Edge): Embeds integrity data into image border pixels via LSB encoding. Hash computation uses `crypto.subtle.digest()` (Web Crypto API on browser/mobile)
2. **Server-side C2PA manifest construction and signing** (`@contentauth/c2pa-node`, runs on Backend): Constructs and signs the C2PA JUMBF manifest box appended to the output PNG

1. Action selection and Digital Source Type (DST):

Action selection is determined by two factors: (a) whether a prior C2PA manifest exists in the input asset, and (b) whether the claim was generated from a verified mobile camera capture (Verified tier, within TOE) or a web/file-pick upload (Standard tier, outside or peripheral to TOE):

Scenario	Primary action	digitalSourceType	TOE
Verified tier, no prior manifest (mobile camera, App Attest/Play Integrity confirmed)	<code>c2pa.created</code>	<code>digitalCapture</code>	Within TOE
Standard tier, no prior manifest (web upload or mobile file pick)	<code>c2pa.opened</code>	None	Web = outside TOE
Any tier, prior C2PA manifest present	<code>c2pa.opened</code>	None	—

`digitalCapture` is asserted **only** when the mobile device hardware (confirmed via App Attest or Play Integrity) directly performed the camera capture. Standard tier uploads use `c2pa.created` with **no digitalSourceType** — OriPics is creating a C2PA manifest for the submitted file but makes no claim about the capture method.

C2PA spec note: `c2pa.opened` requires an ingredient reference (spec constraint); it cannot be used as the first action for uploads with no prior C2PA manifest. `c2pa.created` without `digitalSourceType` is the correct action for Standard tier — it asserts that OriPics created this C2PA manifest, without making any claim about how the underlying image was originally produced.

Implementation: `apps/web/src/lib/oripics-stamp/c2pa.ts (attachC2paManifest())`.

2. Assertions and Claim v2 structure:

OriPics uses `@contentauth/c2pa-node v0.5.x` (backed by `c2pa-rs v0.82.1`) with Claim v2.

Actions are added via `builder.addAction(JSON.stringify(action))` (one call per action). This uses the `builderAddAction` Neon binding which places `c2pa.actions.v2` in `created_assertions` — the conformant location per C2PA Claim v2 spec. (Using `builder.addAssertion('c2pa.actions.v2', ...)` would place it in `gathered_assertions`, which is nonconformant; this approach is not used.)

Custom assertions (`com.oripics.*`) are added via `builder.addAssertion()` and are placed in `gathered_assertions`. This is correct for non-standard vendor assertions.

Assertion label	Claim v2 placement	How added	Content
<code>c2pa.hash.data</code>	<code>created_assertions</code>	Auto-generated by <code>c2pa-rs</code>	Cryptographic hash binding
<code>c2pa.actions.v2</code>	<code>created_assertions</code>	<code>builder.addAction()</code> per action	<code>c2pa.created/c2pa.published/c2pa.opened</code> + platform action
<code>com.oripics.proof</code>	<code>gathered_assertions</code>	<code>builder.addAssertion()</code>	Tier, <code>link_id</code> , <code>verify_url</code> , <code>stamp_version</code> , dimensions, optional GPS
<code>com.oripics.verified</code>	<code>gathered_assertions</code>	<code>builder.addAssertion()</code>	Platform, <code>attest_token_hash</code> , <code>device_integrity</code> (<i>Verified tier only</i>)

3. SCA/SBOM scanning for all content-processing software:

- Same tools as §C.2.3: GitHub Dependabot + CodeQL + npm audit cover all npm dependencies including `@contentauth/c2pa-node` and `pngjs`
- `@contentauth/c2pa-node` is the official C2PA SDK maintained by the Content Authenticity Initiative (CAI). Security updates are tracked via Dependabot
- `pngjs` (used for server-side LSB extraction) is monitored by Dependabot
- Vercel’s immutable deployment model ensures that a specific deployment always runs the exact dependency versions present at build time

4. Vulnerability remediation pipeline:

- Same 90-day CRITICAL/HIGH remediation policy as §C.2.3
- The `@contentauth/c2pa-node` package includes native binary addons (Rust-compiled); Dependabot monitors its npm advisory entries

C.2.5. Protections Against Interception and/or Modification of Traffic

Encryption of Network Traffic

(Required for Assurance Level 1 for Distributed Implementation Class)

TLS versions and cryptographic protocols:

Communication Channel	Protocol	Notes
Edge (all) → Vercel Backend (API calls)	TLS 1.3	Enforced by Vercel CDN/Edge; HTTPS redirect enforced

Communication Channel	Protocol	Notes
Edge (all) → Supabase Storage (PNG upload via signed URL)	TLS 1.3	Supabase enforced
Backend → Supabase (DB + Storage)	TLS 1.3	Enforced via <code>tls.DEFAULT_MIN_VERSION = 'TLSv1.3'</code> in <code>src/instrumentation.ts</code> ; Supabase endpoints support TLS 1.3
Backend → SSL.com eSigner CSC API	TLS 1.3	SSL.com enforced
Backend → SSL.com TSA (timestamp.ssl.com)	TLS 1.3	SSL.com enforced

- **All network communication** between subsystems is encrypted with TLS 1.3
- Vercel enforces TLS 1.3 for all inbound connections; HTTP is automatically redirected to HTTPS
- No plaintext HTTP communication is used between any subsystems
- The `final_hash_hex` transmitted in the JWT chain is protected by TLS in transit and by the HMAC-SHA256 JWT signature at rest

C.2.6. Protections Against Exploitation of Hosting Environment

(Required for Assurance Level 1 for Distributed Implementation Class)

1. IAM system and security boundaries:

- **Vercel:** Team-based RBAC. The OriPics Vercel project is owned by a single team with one owner (Yongseog Son). MFA is required for the team owner account. Vercel enforces isolation between projects and between customers at the infrastructure level (SOC 2 Type II certified)
- **Supabase:** Row-Level Security (RLS) is enabled on all tables. The application uses `service_role` key for server-side operations (never exposed to Edge clients). Edge clients use `anon` key with RLS policies restricting access to authenticated users' own data
- **GitHub:** Repository is private. Branch protection on `main` requires PR approval and CI passage. Force push is disabled
- **Edge clients:** No signing keys, no backend secrets, no `service_role` or `anon` keys embedded in Edge code. Edge clients authenticate users via NextAuth session cookies; all privileged operations require a valid session

2. Access policies for human and non-human principals:

Principal	Access	MFA
Vercel team owner (human)	Full project access, env var management	Yes
Vercel Functions runtime (non-human)	Read env vars, execute functions	N/A
Supabase <code>service_role</code> (non-human)	Full database + storage access	N/A (API key, server-only)
Supabase <code>anon</code> key (non-human)	RLS-restricted read/write	N/A (API key + user JWT)
GitHub repository admin (human)	Source code, CI/CD, secrets	Yes
Dependabot (non-human)	Create PRs for dependency updates	N/A (GitHub-managed)
Edge client / end user (non-human)	NextAuth session only; no infrastructure access	N/A

3. IAM policies for main cloud resources:

- **Vercel Functions:** Only deployed code from the `main` branch can execute in Production. Preview deployments run in isolated environments with separate env vars
- **Supabase Storage** (`oripics-proofs` bucket): Accessed only via `service_role` key from Backend Functions. Signed upload URLs for Edge clients have a 5-minute TTL and are path-restricted to the specific PNG being uploaded. No public write access;

public read is restricted to published links only

- **Supabase PostgreSQL:** All tables have RLS enabled. `service_role` bypasses RLS for server-side operations; anon key is RLS-restricted

4. Vulnerability scanning and security review:

- GitHub Dependabot + CodeQL + Secret Scanning (as described in §C.2.3)
- OWASP Top 10 coverage via CodeQL rules
- Vercel and Supabase perform their own infrastructure-level security management as part of their respective SOC 2 Type II programs

5. Vulnerability remediation process:

- CRITICAL: Target fix within 30 days
- HIGH: Target fix within 90 days
- MODERATE: Target fix within 180 days
- LOW: Tracked and addressed on a best-effort basis
- Dependabot automatically creates PRs; team triages within 7 days
- Evidence of historical remediation: 21 advisories triaged 2026-05-09, Next.js security patch applied same day

C.2.7. Capture-Only Content Policy and Pre-Existing Manifest Handling

Content Policy and TOE Boundary

OriPics is a **photo provenance platform**. The `digitalCapture` DST is asserted only when the Generator Product can cryptographically confirm that the asset was captured by a hardware camera under the control of an authenticated device:

Client	Tier	TOE	Action	DST
OriPics Web	Standard	Outside TOE	<code>c2pa.created</code> (no DST)	None
OriPics iOS	Standard (file pick)	In TOE (device auth'd)	<code>c2pa.created</code> (no DST)	None
OriPics iOS	Verified (camera capture)	In TOE (App Attest)	<code>c2pa.created</code>	<code>digitalCapture</code>
OriPics Android	Standard (file pick)	In TOE (device auth'd)	<code>c2pa.created</code> (no DST)	None
OriPics Android	Verified (camera capture)	In TOE (Play Integrity)	<code>c2pa.created</code>	<code>digitalCapture</code>

OriPics does not certify AI-generated images, screenshots, or composites. Claims of `digitalCapture` are issued only when device attestation (App Attest / Play Integrity) confirms in-app camera hardware capture.

Pre-Existing C2PA Manifest Handling

When an uploaded image contains a pre-existing C2PA manifest (e.g., a photo previously signed by another C2PA generator product such as a Pixel Camera, Leica M11-P, or another conformant tool), OriPics detects this using `@contentauth/c2pa-node Reader.fromAsset()` before constructing the new manifest.

Implemented: Approach A — Ingredient (`parentOf`)

OriPics implements **Approach A**: when a pre-existing C2PA manifest is detected, the prior manifest is preserved as a `c2pa.ingredient` assertion (`relationship: "parentOf"`) in the new OriPics manifest. This maintains the full provenance chain.

Implementation (`apps/web/src/lib/oripics-stamp/c2pa.ts`):

1. Before constructing the new manifest, `Reader.fromAsset()` attempts to read the active manifest from the input PNG
2. If a prior manifest is found:
 - The primary action is set to **`c2pa.opened`** (C2PA spec §9.3.2; indicates the content was opened from an existing certified source, not created from scratch)

- `builder.addIngredient()` is called with `relationship: "parentOf"` to chain the prior manifest

3. If no prior manifest is found:

- The primary action is **`c2pa.created`** with `digitalSourceType: digitalCapture`
- No ingredient is added

Scenario	Action	Ingredient
Verified tier, mobile camera, no prior C2PA	<code>c2pa.created + digitalCapture DST</code>	None
Standard tier, web/file-pick, no prior C2PA	<code>c2pa.created (no DST)</code>	None
Any tier, prior C2PA manifest present	<code>c2pa.opened (no DST)</code>	<code>c2pa.ingredient.v3</code> with <code>relationship: "parentOf"</code>

(Note on Verified tier): Mobile native camera captures (Verified tier) produce fresh in-app captures with no prior C2PA manifest, so the ingredient path primarily applies to Standard tier uploads of already-certified images.

Attachments

The following sample output files are provided with this submission:

1. **sample-oripics-standard.png** — Sample PNG with OriPics C2PA manifest (Standard tier, sandbox cert). Action: `c2pa.created` with `digitalCapture DST`. No prior manifest.
2. **sample-oripics-standard-manifest.json** — Extracted C2PA manifest JSON from the above sample.
3. **sample-oripics-with-ingredient.png** — Sample PNG demonstrating §C.2.7 Approach A: a Google Pixel Camera image (Unedited regular capture.jpg from the C2PA Interoperability Testing Library) processed by OriPics. Action: `c2pa.opened`. Prior Pixel Camera manifest included as `c2pa.ingredient.v3` with `relationship: "parentOf"`.
4. **sample-oripics-with-ingredient-manifest.json** — Extracted C2PA manifest JSON from the above sample.

(Note: Samples are signed with a sandbox test certificate. The manifest structure is identical to production; only the signing certificate changes when production credentials are issued from SSL.com.)